

Deep Reinforcement Learning Algorithm for Latency-Oriented IIoT Resource Orchestration

Peiyang Zhang^{1b}, *Member, IEEE*, Yi Zhang^{1b}, Neeraj Kumar^{2b}, *Senior Member, IEEE*,
and Ching-Hsien Hsu^{3b}, *Senior Member, IEEE*

Abstract—Due to geographical factors and resource constraints, the traditional Internet architecture cannot meet the needs of the space-air-ground-integrated network (SAGIN) resource layout in the Industrial Internet of Things (IIoT) service. How to arrange network resources in SAGIN quickly and efficiently to meet the quality of service requirements of users has become a hot research topic in the industry. Based on the characteristics of SAGIN with multiple network segments, we convert the resource scheduling problem of SAGIN into a multidomain virtual network embedding (VNE) problem. This article proposes a latency-sensitive VNE algorithm based on deep reinforcement learning (DDRL-VNE) in the SAGIN environment. Unlike traditional latency optimization algorithms, we consider the effect of traffic size and hop count on latency when evaluating latency. We constructed a learning agent composed of a five-layer policy network and extracted a feature matrix as its training environment based on the network attributes of SAGIN. The node embedding is completed according to the probability that each node is embedded in the training, and then the breadth-first search strategy is used to complete the link embedding. The

experimental results effectively illustrate the effectiveness of the algorithm in the SAGIN resource allocation problem.

Index Terms—Deep reinforcement learning (DRL), Industrial Internet of Things (IIoT), latency, space-air-ground-integrated network (SAGIN).

I. INTRODUCTION

THE COMPLETION of the formulation of the 5G network standard also means that the research on the 6G network architecture needs to be further deepened to promote the development of the Industrial Internet of Things (IIoT) [1], [2]. IIoT devices (such as robots and sensors) are widely distributed around the world and may appear anywhere on land, sea, or air. However, traditional terrestrial communication networks have shortcomings, such as the easy destruction of the ground infrastructure and limited signal coverage [3], [4]. Therefore, in the future IIoT application scenarios, the space-air-ground-integrated network (SAGIN) will be used to support IIoT and achieve full-scale wireless coverage [5], [6]. SAGIN is flexible and is not restricted by the geographical environment. As shown in Fig. 1, it can provide safe, reliable, and low-latency communication services for various application scenarios in IIoT [7]. Quality-of-Service (QoS) properties, such as cost, revenue, security, and latency, are widely used to measure the nonfunctionalities of a service [8], [9]. With the development of IIoT, the demand for networks is gradually changing from meeting the required functions to the differentiated demand for QoS [10], [11]. For example, terrain scanning needs the lowest possible transmission cost; intelligent vehicle networks need low-latency network guarantees; online classrooms need network support with high-bandwidth resources. In addition, in a large-scale and dynamic network environment like SAGIN, IIoT user satisfaction is more sensitive to network latency.

However, the network topology of the SAGIN is constantly changing, with strong time-varying and self-organizing characteristics, which brings severe challenges to the flexible scheduling of heterogeneous network resources and the realization of low-latency transmission [12]. Network virtualization (NV) technology breaks through traditional hardware constraints, can significantly improve network flexibility, customization, and agility, and can well solve the problem of network resource scheduling in SAGIN [12]. We can use NV technology to realize the abstraction, fragmentation, and distribution of the underlying physical network [13]. The core problem of NV technology is virtual network embedding (VNE), and its essence is to allocate these underlying

Manuscript received 27 January 2022; revised 31 August 2022 and 23 November 2022; accepted 8 December 2022. Date of publication 14 December 2022; date of current version 7 April 2023. This work was supported in part by the Shandong Provincial Natural Science Foundation, China, under Grant ZR2020MF006 and Grant ZR2022LZH015; in part by the Industry-University Research Innovation Foundation of Ministry of Education of China under Grant 2021FNA01001; in part by the Major Scientific and Technological Projects of CNPC under Grant ZD2019-183-006; in part by the Open Foundation of State Key Laboratory of Integrated Services Networks (Xidian University) under Grant ISN23-09; and in part by the Open Foundation of State Key Laboratory of Networking and Switching Technology (Beijing University of Posts and Telecommunications) under Grant SKLNST-2021-1-17. (Corresponding authors: Neeraj Kumar; Peiyang Zhang.)

Peiyang Zhang and Yi Zhang are with the College of Computer Science and Technology, China University of Petroleum (East China), Qingdao 266580, China, and also with the State Key Laboratory of Integrated Services Networks, Xidian University, Xi'an 710071, China (e-mail: zhangpeiyang@upc.edu.cn; zhangyi.upc@qq.com).

Neeraj Kumar is with the Department of Computer Science and Engineering, Thapar Institute of Engineering and Technology (Deemed to be University), Patiala 147004, India, also with the School of Computer Science, University of Petroleum and Energy Studies, Dehradun 248007, India, also with the Faculty of Computing and Information Technology, King Abdulaziz University, Jeddah 21589, Saudi Arabia, and also with the Department of Electrical and Computer Engineering, Lebanese American University, Beirut 1102 2801, Lebanon (e-mail: nehra04@gmail.com; neeraj.kumar@thapar.edu; neeraj.kumar@lau.edu.lb).

Ching-Hsien Hsu is with the Department of Computer Science and Information Engineering, Asia University, Taichung 41354, Taiwan, also with the Department of Medical Research, China Medical University Hospital, China Medical University, Taichung 40402, Taiwan, and also with the Guangdong-Hong Kong-Macao Joint Laboratory for Intelligent Micro-Nano Optoelectronic Technology, School of Mathematics and Big Data, Foshan University, Foshan 528000, China (e-mail: robertchh@gmail.com).

Digital Object Identifier 10.1109/IIOT.2022.3229270

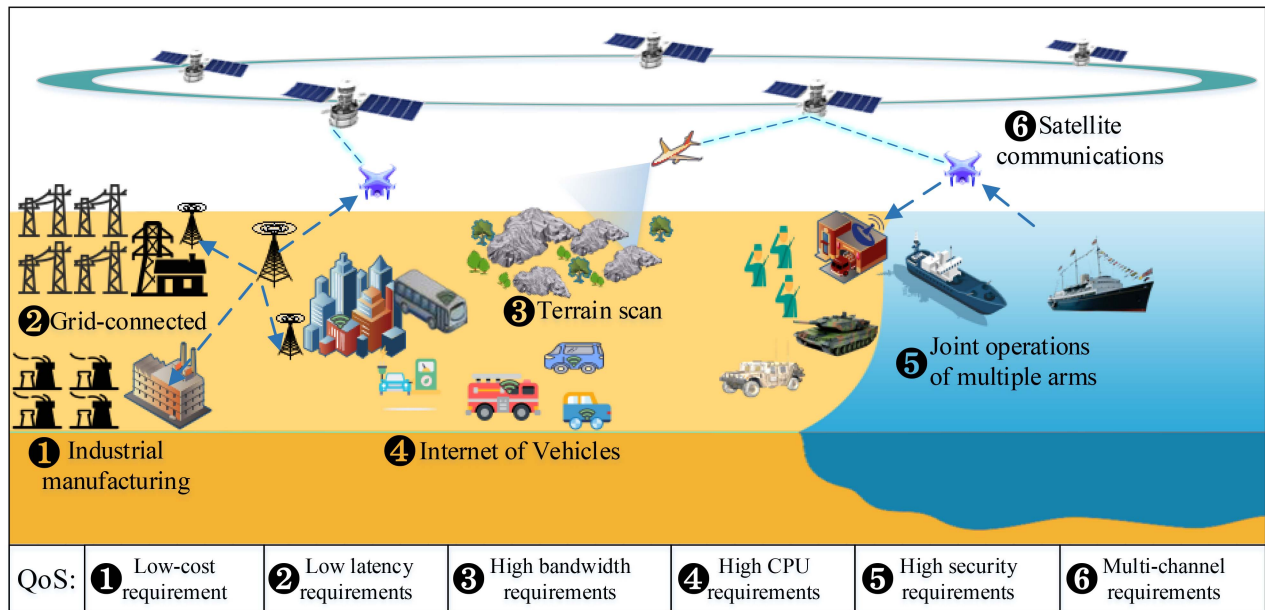


Fig. 1. Wireless communication in SAGIN scenario.

resources reasonably according to the needs of the application scenario. Therefore, we can solve the resource allocation problem of IIoT by studying the multidomain VNE algorithm [14], [15].

The VNE problem has been proved to be an NP-hard problem. When using a heuristic algorithm to solve it, most of them rely on artificial rules to rank nodes or make mapping decisions, and it is not easy to find the optimal solution. The parameter settings of these algorithms are fixed and the algorithm complexity is high. In addition, existing VNE algorithms usually consider only the link length when calculating the network latency, and simply assume that the response time is proportional to the link length. However, in actual situations, due to the influence of materials and hop counts, the propagation latency of the link cannot be evaluated directly by length. In the SAGIN, the link latency between different network segments is more different, which requires the improvement of the existing VNE algorithm. In this case, a latency calculation method that can comprehensively consider network traffic and hops is critical. In recent years, deep reinforcement learning (DRL) have shown a powerful ability to solve high-dimensional spatial decision-making problems and have achieved good results in many fields [16], [17]. DRL can be used to perceive and make decisions on the problem scene, and it has strong versatility [18], [19]. DRL has better performance than heuristic optimization methods in solving NP-hard problems like VNE [20]. Therefore, we choose to apply DRL to meet the differentiated QoS and latency needs of SAGIN in IIoT [21], [22].

The main work of this article is as follows.

- 1) We propose a latency-aware multidomain VNE algorithm named DDRL-VNE for latency-oriented IIoT services in SAGIN. The algorithm solves the network resource allocation problem in IIoT from the perspective of VNE, optimizes for the impact of link traffic and hop count on latency, and reduces the VNE embedding latency.

- 2) We introduce DRL in the node embedding stage to improve the quality of node selection. Specifically, we abstract SAGIN into three network segments and devise an artificial neural network as an agent. The network attributes of the SAGIN network segment constitute a feature matrix, which is used as the input of the self-built policy network to deduce the node embedding probability.
- 3) Unlike the previous VNE algorithm, we introduce traffic and hop count constraints in the link embedding stage to reduce the overall latency of the VNE solution. Simulation experiment results show that our proposed algorithm performs well in multiple network performance.

The remainder of this article is organized as follows. In Section II, we summarize the research work related to IIoT. In Section III, we introduced SAGIN-based VNE-related issues and established a system model. In Section IV, we introduce our proposed algorithm and implementation process in detail. In Section V, we discussed the results of simulation experiments. In Section VI, we summarize the work in this article.

II. RELATED WORKS

The network communication technology of the IIoT has always been a hot spot in the research field. SAGIN can provide a wide range of communication services, which has a significant impact on the development of IIoT. A brief summary of some of the relevant works in SAGIN and VNE is presented in this section.

A. Research Related to SAGIN in IIoT

The SAGIN covers a wide range and can provide connection services for a large number of network devices. How to allocate network resources reasonably and efficiently

orchestrate connections between massive devices has become an urgent problem to be solved [23], [24].

Zhou et al. [12] used the Markov decision process to solve the task scheduling problem of the IoT in SAGIN and proposed a risk-sensitive reinforcement learning algorithm. This method significantly reduces the latency of IoT computing tasks. Ding et al. [21] proposed a new algorithm for safely switching user networks in SAGIN. The algorithm is based on the software-defined networking (SDN) architecture and optimizes the selection of satellite nodes to reduce latency. Document [25] proposed a method to solve the IoT flow calculation problem by using SAGIN. This method used DRL to learn the optimal flow decision, which could efficiently allocate network resources and schedule load tasks.

Li et al. [26] described the current challenges of maritime communication networks and proposed a solution to improve maritime communication performance by using UAVs to provide dynamic on-demand coverage at sea. However, this approach is unsafe when drones are far from shore and can lead to high costs and latency. Blockchain technology can solve the security problems in SAGIN, but it will bring huge latency and storage resource occupation. To this end, Sun et al. [27] proposed a collaborative blockchain architecture that uses multiple parallel blockchains to orchestrate SAGIN's network resources and uses relay chain technology to ensure collaboration security.

The above article discusses various ways to optimize SAGIN performance. Inspired by these, we solve the problem of SAGIN resource allocation in IoT from the perspective of network embedding.

B. VNE Related Research

The emergence of SDN provided a platform for the application and promotion of VN [28]. Li et al. [29] proposed a self-adaptive VNE algorithm. The VN request is divided into different types through an adaptive algorithm, and the VNE problem is solved by the integer linear programming method. Chai et al. [30] studied the VNE problem in SDN where the substrate SDN switches and links may be subject to malicious attacks. They proposed a hierarchical virtualization-enabled SDN architecture based on which the VNE strategy can be designed to minimize network load and maximizes embedding reliability.

Rui-Wen and Han-Cheng [31] proposed a new embedding scheme for multidomain SDN virtual networks, which could significantly reduce the overall embedding overhead and possess excellent stability and feasibility. Wu et al. [32] proposed an SDN-based framework for the Internet of Vehicles in the SAGIN environment. The framework uses the complementary advantages of different network segments to achieve reliable and efficient resource scheduling.

Most of the existing researches on optimizing the performance of IIoT network services consider aspects, such as security and energy consumption [33], and few papers study the issue of latency. In IIoT systems, the impact of latency on user experience cannot be ignored. Based on SAGIN, we solve the service delay problem from the perspective of

virtual network architecture. The traditional heuristic-based VNE algorithm is easy to fall into the local optimal solution, while DRL has better performance due to its powerful learning ability [34], [35]. To this end, this article analyzes the architecture of SAGIN and implements a multidomain VNE algorithm based on DRL.

III. DESCRIPTION AND MODEL ESTABLISHMENT OF SYSTEM

In this chapter, we give a detailed introduction to the VNE problem of IIoT in SAGIN, including the definition, solution, and formal description of related conditions.

A. Problem Description

The VNE mainly solves how to dynamically map the virtual network to the physical network to maximize the advantages of the hardware. We can think of SAGIN as a hierarchical network architecture consisting of three network segments. The air nodes of SAGIN are in motion at all times and have time-varying dynamic topological characteristics. Different network segments have different link latency properties at different times. Therefore, we cannot directly migrate the VNE algorithm of the terrestrial network to SAGIN. To solve this problem, we use the idea of virtual topology strategy, continuous time-varying discrete dynamic network topology into N time slices, and approximated that the network topology in each time slice T is static. Then, we can transform the resource allocation problem in SAGIN into a multidomain VNE problem.

B. Problem Modeling

1) *Physical Network Model*: The physical network is modeled as a weighted undirected graph $G^S = \{N^S, L^S, A_N^S, A_L^S, T\}$, where G^S represents the underlying network of SAGIN, N^S represents the collection of physical nodes, L^S represents the collection of physical links, A_N^S represents the collection of node attributes, A_L^S represents the collection of link attributes, and assume that the network topology remains unchanged during the time interval T . There are three types of nodes in the physical node collection N^S , which are space-based nodes, air-based nodes, and ground-based nodes. The physical link collection L^S contains a variety of links, including intradomain links in three different network segments, and interdomain links between different network segments. The network node attribute set A_N^S in SAGIN includes the amount of available CPU resources of the physical node, etc. The network link attribute set A_L^S in SAGIN includes the available bandwidth resource amount and latency attribute of the physical link.

2) *Virtual Network Model*: The virtual network is also modeled as a weighted undirected graph $G^V = \{N^V, L^V, A_N^V, A_L^V, \Delta t\}$, where G^V represents a virtual network, N^V represents a collection of virtual nodes, A^V represents the attribute set of VNRs, and Δt represents the duration of the virtual network. The network node attribute set A_N^V in the VNR includes the available CPU resource CPU_N^V of the virtual node. The network link attribute set A_L^V

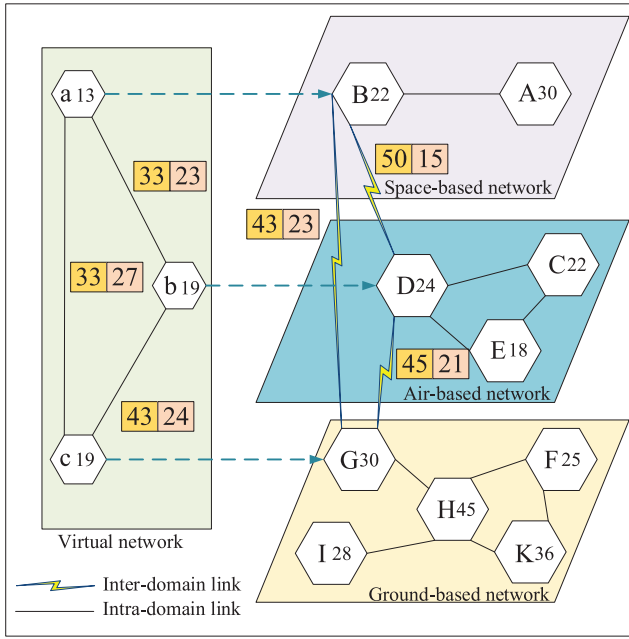


Fig. 2. Physical network, virtual network, and possible embedding examples.

in the VNR includes the available bandwidth resource BW_L^V and the delay attribute D_L^V of the virtual link.

3) *Virtual Network Embedding Model*: The multidomain VNE process can be represented by VNE: $G^S \rightarrow G^V$. Fig. 2 shows the embedding process of a multidomain virtual network in SAGIN. The hexagon in the figure represents network nodes, and the line connecting two nodes represents network links. The numbers inside the hexagon indicate the CPU capacity of the physical node and virtual node, respectively, and the numbers next to the link indicate the number of bandwidth resources and the delay value, respectively. The figure shows a possible embedding result of VNR in SAGIN, virtual nodes a, b, and c are mapped to the space-based node B, the air-based node D, and the ground-based node G, respectively. The process of VNR embedding needs to satisfy specific resource requirements and constraints. For example, if node b is mapped to E, the computational resources required by node b cannot be satisfied.

As shown in (1), in the embedding process, the resources required by the virtual request node must be less than or equal to the available resources of the physical node of the underlying network

$$\text{CPU}(N_i^v) \leq \text{CPU}(N_j^s) \quad (1)$$

where $\text{CPU}(N_i^v)$ represents the CPU required resource amount of the virtual node.

Similarly, the bandwidth resource requirement of a virtual link should be less than or equal to the bandwidth available resource of a physical link, as shown in

$$\text{BW}(N_m^v, N_n^v)^v \leq \text{BW}(N_i^s, N_j^s)^s \quad (2)$$

where $\text{BW}(N_m^v, N_n^v)^v$ represents the amount of bandwidth resource required by virtual link.

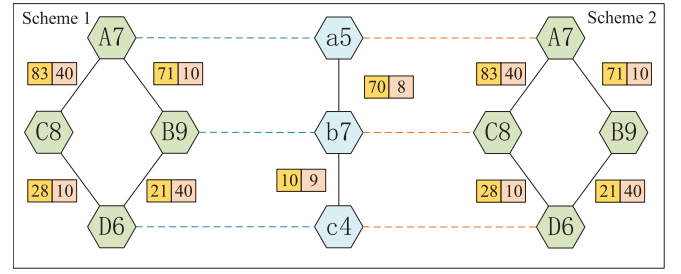


Fig. 3. Two embedding schemes of VN.

Although the SAGIN covers a wide area, the orbital height of its satellite nodes is too high, resulting in a corresponding increase in the transmission latency in the network. Link latency in different network segments is different, and the latency of interdomain links is often greater than that of intradomain links. For this reason, we set the latency attribute for the link, and the latency of the physical link must be less than or equal to the latency requirement of the virtual link, as shown in the following formula:

$$D(N_m^v, N_n^v)^v \leq D(N_i^s, N_j^s)^s. \quad (3)$$

4) *Performance Indicators*: The potential traffic demand of the SAGIN depends on the traffic transmission intensity within the network coverage area, which is mainly affected by geographical location and time changes. To pursue the overall minimum latency, we have considered the basic latency constraints and the influence of link traffic on the latency when evaluating the overall latency. We use the bandwidth requirement of the link to correspond to the traffic size of the link and use (4) to represent the latency performance of the VNE result

$$D(G^V) = \sum \text{BW}(N_m^v, N_n^v)^v \times H(N_m^v, N_n^v)^v \times D(N_i^s, N_j^s)^s \quad (4)$$

where $D(G^V)$ represents the overall latency performance indicators of the virtual network G^V embedding scheme, $H(N_m^v, N_n^v)^v$ represents the number of hops of virtual link (N_m^v, N_n^v) , and $D(N_i^s, N_j^s)^s$ represents the latency attribute of the physical link (N_i^s, N_j^s) . The smaller the value of $D(G^V)$, the better the latency performance of the embedding scheme.

To better illustrate how to minimize the overall latency, we give an example. Fig. 3 shows the two embedding schemes of VNR. The number next to the physical link represents the latency attribute of the physical link, and the number next to the virtual link represents the traffic size of the virtual link. In scheme 1, the latency performance indicators of virtual network G^V is $D(G^V) = 70 \times 1 \times 10 + 10 \times 1 \times 40 = 1100$. In the second scheme, the latency performance indicators of the virtual network G^V is $D(G^V) = 70 \times 1 \times 40 + 10 \times 1 \times 10 = 2900$. Both schemes meet the embedding constraints, but from the perspective of network operation, the overall delay of scheme 2 is close to three times that of scheme 1.

In addition to the overall latency, this article also evaluates the performance of the multidomain VNE algorithm under

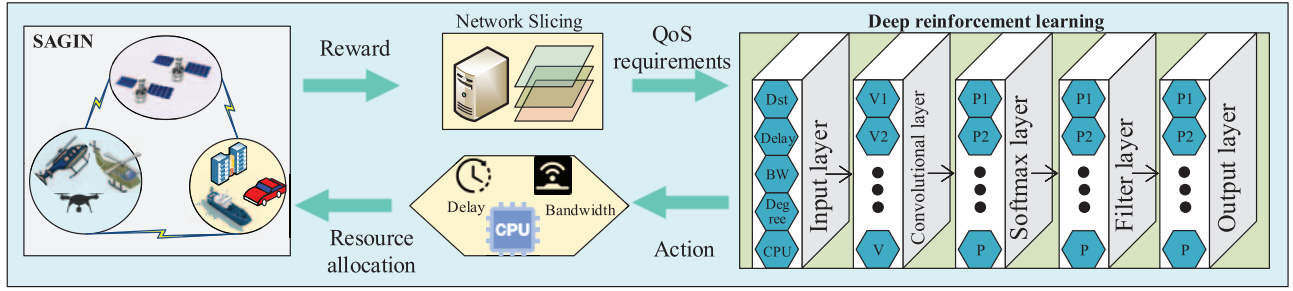


Fig. 4. DRL in SAGIN.

SAGIN from three aspects: long-term average revenue, long-term average revenue-cost ratio, and virtual request acceptance rate. The goal of a multidomain VNE is to receive as much VNR as possible to obtain more benefits. We use (5) to calculate the revenue of VNE

$$R(G^V, \Delta t) = \Delta t \cdot \sum_{N_i^V}^{N^V} \text{CPU}(N_i^V) + \Delta t \cdot \sum_{(N_m^V, N_n^V)}^{L^V} \text{BW}(N_m^V, N_n^V) \quad (5)$$

where Δt represents the duration of the virtual network, $\text{CPU}(N_i^V)$ reveals the computing resources required by the virtual nodes, and $\text{BW}(N_m^V, N_n^V)$ is the bandwidth resources required by the virtual links. As the formula shows, the revenue of virtual network requests in SAGIN is proportional to the amount and duration of requested resources.

As shown in (6), we use the node's CPU and link bandwidth resource consumption to calculate the cost of VNE

$$C(G^V, \Delta t) = \Delta t \cdot \sum_{N_i^V}^{N^V} \text{CPU}(N_i^V) + \Delta t \cdot \sum_{(N_m^V, N_n^V)}^{L^V} \text{BW}(N_m^V, N_n^V) \cdot \text{hops}[(N_m^V, N_n^V)] \quad (6)$$

where $\text{hops}[(N_m^V, N_n^V)]$ denotes the number of hops of the virtual link (N_m^V, N_n^V) .

Long-term average revenue is an important indicator to measure the performance of the VNE algorithm, which can be defined as follows according to (5):

$$\lim_{T \rightarrow \infty} \frac{\sum_{\Delta t \in T} R(G^V, \Delta t)}{T}. \quad (7)$$

The higher the long-term revenue-cost ratio, the greater the revenue generated. According to (5) and (6), the long-term average revenue-cost ratio can be defined as follows:

$$\lim_{T \rightarrow \infty} \frac{\sum_{\Delta t \in T} R(G^V, \Delta t)}{\sum_{\Delta t \in T} C(G^V, \Delta t)}. \quad (8)$$

We define the acceptance rate as follows:

$$\lim_{T \rightarrow \infty} \frac{\sum_{\Delta t \in T} G_{acc}^V}{\sum_{\Delta t \in T} G_{arr}^V} \quad (9)$$

where G_{acc}^V represent the VNR successfully accepted within T time, and G_{arr}^V represent the VNR arriving within T time.

IV. ALGORITHM IMPLEMENTATION

In this section, we will introduce the VNE algorithm based on DRL. As shown in Fig. 4, we extracted the five attributes of the substrate nodes to form a feature matrix to construct a training environment for the agent. In the node embedding stage, the agent is trained to get the probability of the physical node being embedded, and the shortest path algorithm is used to complete the link embedding. The agent adjusts its own action strategy based on the rewards it receives, thus discovering a better VNE solution.

A. Feature Extraction

When applying the DRL method to node embedding, it is first necessary to convert the original attributes of the physical node into a set of features with obvious statistical significance. We extracted five network attributes for nodes based on the differentiated service quality requirements in SAGIN and used them as input to the policy network. The five characteristics of nodes are computing resources, degrees, the sum of link bandwidths, the sum of link latencies, and the average distance to other unembedded nodes.

Computing resources determine the upper limit of the number of virtual nodes that a physical node can carry. Virtual nodes will preferentially be embedded on physical nodes with abundant available CPU resources. In SAGIN, the degree of a physical node N^S represents the total number of intradomain links and interdomain links connected to it. Nodes with a high amount of link connections are usually more likely to be connected by other physical nodes to carry the embedded virtual link. The sum of the link bandwidth is the total amount of available bandwidth resources of the links adjacent to the physical node N^S . The larger the value, the more candidate links are available for embedding when the virtual node is mapped to N^S . Similar to the sum of bandwidth, the sum of link latency refers to the total amount of latency of links adjacent to the physical node N^S . We evaluate the distance between the origin node and the arriving node of the link based on the number of hops of the shortest path. In the node embedding stage, preferentially select physical nodes close to

the embedded physical nodes, which can effectively reduce link bandwidth costs and link latency

$$\text{SUM}(N^S)_{bw} = \sum_{(N_m^S, N_n^S) \in L_N^S} \text{BW}(N_m^S, N_n^S) \quad (10)$$

where $\text{SUM}(N^S)_{bw}$ represents the total bandwidth of adjacent links of node N^S , L_N^S represents the set of links connected to node N^S , and $\text{BW}(N_m^S, N_n^S)$ represents the available bandwidth resource of link (N_m^S, N_n^S) .

Similar to the above formula, the calculation method of the sum of the latency of the adjacent links of a node is as follows:

$$\text{SUM}(N^S)_d = \sum_{(N_m^S, N_n^S) \in L_N^S} D(N_m^S, N_n^S). \quad (11)$$

Contrary to the sum of bandwidth, the smaller the value of the adjacent link latency of a physical node, the more likely it is to be selected. We evaluate the distance between the origin node and the arriving node of the link based on the number of hops of the shortest path. In the node embedding stage, preferentially select physical nodes close to the embedded physical nodes, which can effectively reduce link bandwidth costs and link latency. We use the following formula to calculate the average distance between a node and other embedded nodes

$$\text{AVG}(N^S)_{\text{dst}} = \frac{\sum_{N_i^S \in N_{\text{non}}^S} \text{dst}(N^S, N_i^S)}{|N_{\text{non}}^S| + 1}. \quad (12)$$

In (12), N_{non}^S represents those physical nodes that have been embedded by virtual nodes, and $\text{dst}(N^S, N_i^S)$ represents the distance from node N^S to node N_i^S .

In fact, the network environment of SAGIN is very complicated, and we can extract more features from the substrate network information to improve the performance of the learning agent. However, the more features are extracted, the complexity of the algorithm will increase accordingly. We have considered the practical application of SAGIN for differentiated service quality and have selected the five features mentioned above. After extracting the feature value of the node, we normalize it and transform it into the feature vector as shown below

$$[\text{CPU}(N_k^S), \text{DEG}(N_k^S), \text{SUM}(N_k^S)_{bw}, \text{SUM}(N_k^S)_{\text{delay}}, \text{AVG}(N_k^S)_{\text{dst}}].$$

B. Policy Network

In the DDRL-VNE, the agent observes the current state, takes actions following the established policy, and finally receives instant returns and the latest state, and optimizes and adjusts the policy. The agent will repeat this process until it finally obtains the optimal policy. We use the basic elements of the neural network to form a five-layer policy network to act as agent agents. They are the extraction layer, the convolution layer, the probability layer, the filter layer, and the output layer.

In the input layer, we extract the feature matrix from the network information of SAGIN as the input of the agent. The convolution kernel in the convolution layer will perform convolution operations based on the feature matrix to evaluate the

Algorithm 1 Training Process

Input: G_S, G_V , epoch, Learning rate, α , Training set;

Output: Probability of SAGIN nodes being embedded;

```

1: Random initialize policy network parameters;
2: while iteration < epoch do
3:   for request  $\in$  trainingSet do
4:     for  $n_v \in$  request do
5:       getFeatureMatrix() // Extract the feature matrix
        of the substrate network;
6:       getProbability() // Output the probability that
        the physical node is embedded;
7:     end for
8:     if isMapped( $\forall N_v \in G_V$ ) then
9:       Virtual links embedding;
10:    end if
11:    if isMapped( $\forall N_v \in G_V, \forall L_v \in G_V$ ) is false then
12:      Calculate reward;
13:    else
14:      Clear Gradient;
15:    end if
16:  end for
17:  iteration++;
18: end while

```

available resources of each base node. The calculation method of convolution is as follows:

$$h_k^c = w \cdot v_k + b \quad (13)$$

where h_k^c is the k th output of the convolutional layer, w is the weight vector of the convolution kernel, v_k is the k th input of the convolutional layer, and b is the bias. On the probability layer, the softmax function is used to find the probability of different embedding results. The probability calculation formula is as follows:

$$P_i = \frac{e^{h_k^c}}{\sum_j e^{h_j^c}} \quad (14)$$

where P_i represents the possibility of the physical node being embedded. After that, the filtering layer filters out nodes whose available CPU resources cannot meet the embedding constraints. Finally, the embedding probability of each available physical node is output in the output layer and sorted according to the probability.

C. Training and Testing

Algorithm 1 shows the training process of DDRL-VNE. Lines 5–7 show the node embedding stage, and lines 8–10 show the link embedding stage. At the beginning of the training phase, the parameters in the training network need to be initialized randomly and adjusted through multiple pieces of training. Then, extract a feature matrix according to the attributes of the nodes in SAGIN, and use it as the input when the new VNR arrives. The policy network will filter out the available physical nodes according to the embedding constraints and output the probability of each physical

Algorithm 2 Testing Process

Input: Testing set;
Output: latency, acceptance ratio, long-term average revenue-cost ratio, long-term average revenue;
1: Initialize the parameters of the policy network;
2: **for** request $\in$ testSet **do**
3: **for** $n_v \in$ request **do**
4: Virtual nodes embedding;
5: **end for**
6: Using BFS strategy to find the shortest path;
7: Virtual links embedding;
8: **if** isMapped($\forall n_v \in G_V, \forall L_v \in G_V$) **then**
9: return success;
10: **end if**
11: **end for**

node being embedded. The higher the probability, the greater the possibility of selecting this node for embedding to produce good results. We generate a sample of available physical nodes according to the probability distribution and select the physical nodes that will eventually be embedded. After that, the breadth-first search strategy is used to complete the link embedding. We use the ratio of revenue to latency as the agent's reward signal, which can comprehensively reflect the utilization of network resources and network latencies. The learning agent needs to adjust its behavior according to the reward signal. A higher reward value can make the agent know that its current behavior is effective. When the reward value obtained by the agent is small, it indicates that it should adjust its current behavior in time.

In the testing phase, we directly select nodes for embedding based on the embedding probability output by the policy network. The completion process of link embedding is the same as the training phase. The testing process is shown in Algorithm 2.

D. Complexity Analysis

The training phase of the DRL agent is performed online and the testing phase is offline. Therefore, we analyze the complexity of the training phase as the complexity of the DDRL-VNE algorithm. The algorithm extracts five network attributes of the nodes to form a $5 \times N$ feature matrix. Therefore, the time complexity of extracting features from the underlying network is $O(N)$, and the time complexity of solving all feature vectors is $O(N^2)$. The arrival of a new VNR causes the feature matrix to be updated, so the time complexity of updating the feature matrix is $O(KN^2)$. The final complexity of the algorithm is $O(KN^2 + N^2 + N)$, where N is the total number of physical nodes and K is the number of nodes in the VNR that are successfully embedded.

V. ANALYSIS OF RESULTS

In this section, we will introduce the design details of the experiment in detail and analyze the experimental results.

TABLE I
SETTINGS OF PARAMETERS

Parameter Items	The Range
Physical nodes	100
Physical links	500
Space-based network nodes	10
Air-based network nodes	25
Ground-based network nodes	65
Computing resources of space-based network nodes	U[20,40]
Bandwidth resources of space-based network links	U[50,100]
Delay values of space-based network links	U[30,40]
Computing resources of air-based network nodes	U[20,40]
Bandwidth resources of air-based network links	U[60,100]
Delay values of air-based network links	U[20,40]
Computing resources of ground-based network nodes	U[50,100]
Bandwidth resources of ground-based network links	U[50,100]
Delay values of ground-based network links	U[10,20]
Number of virtual nodes	U[2,10]
Computing resource required by virtual nodes	U[0, 50]
Bandwidth resources required by virtual links	U[0, 50]
Delay requirements of virtual links	U[1,25]
Bandwidth resources of inter-domain links	U[50,100]
Delay values of inter-domain links	U[30, 60]

A. Environmental Simulation and Parameter Setting

In order to comprehensively evaluate the performance of the algorithm, we used the GT-ITM tool to generate a three-layer network topology composed of 100 physical nodes and about 500 physical links to simulate the SAGIN environment. Space-based networks and air-based networks have wider coverage, but the network latency is larger. The terrestrial network can provide low-latency services, but the network coverage is limited.

In order to simulate the real SAGIN environment as much as possible, we divide the 100 physical nodes into three types. Subsequently, 2000 VNRs containing 2–10 virtual nodes were randomly generated, of which 1000 were used as training samples and the rest were used as test samples. The network segment where each virtual node is embedded is randomly selected, and virtual links can only be embedded in physical links that meet the constraints of bandwidth resources and latency requirements. There is an average of four VNRs within 100-time units, and the duration of each request follows an exponential distribution of an average of 100-time units. The parameter settings of the network properties are shown in Table I.

B. Experimental Results and Analysis

Experiment 1 (Training Result Analysis): In DRL, the learning agent needs to go through a training stage to make the algorithm performance relatively stable. We collected the results of 100 epochs of the algorithm on the training set and observed and analyzed its performance. Fig. 5 shows the change in long-term average revenue, long-term revenue-cost ratio, acceptance rate, and cumulative latency of the agent during the training process. At the beginning of the training, the agent was not familiar with the SAGIN environment, so the performance was poor. As training progresses, the agent will explore various possible solutions. As training progresses, the agent explores various possible solutions. The

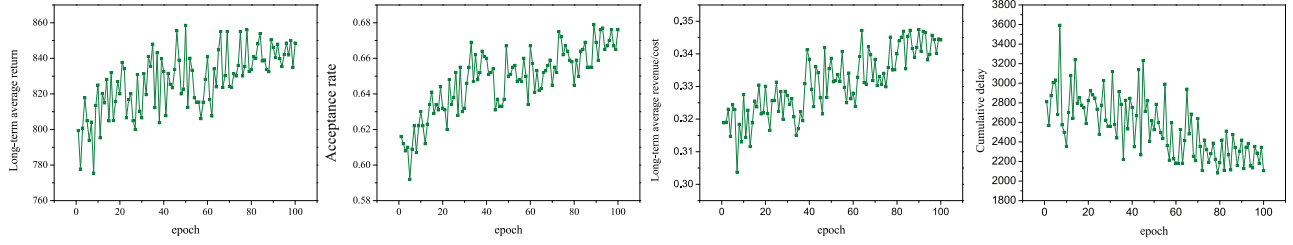


Fig. 5. Training process.

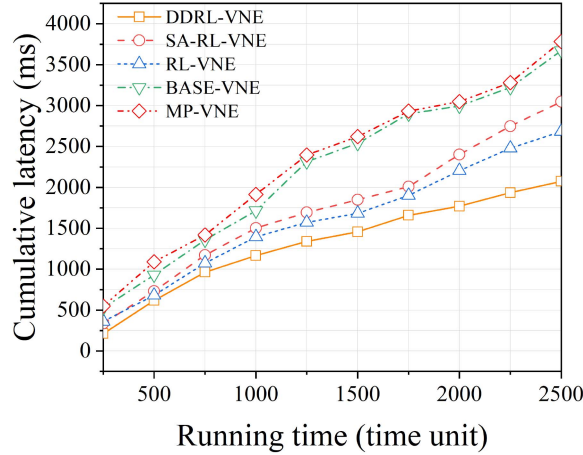


Fig. 6. Comparison of cumulative latency.

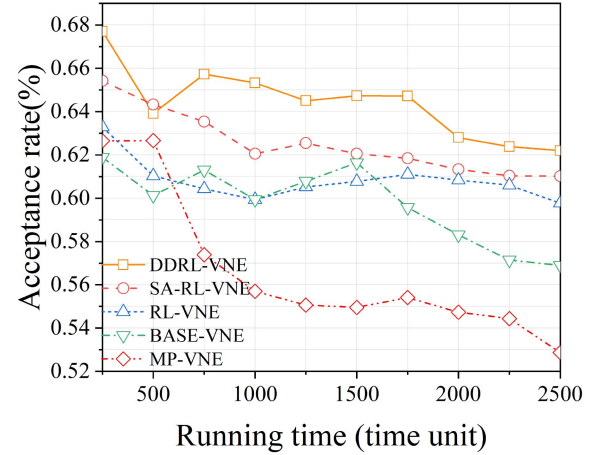


Fig. 7. Comparison of VNRs acceptance rate.

agent sometimes makes wrong choices when exploring solutions, so performance fluctuates during training. When the agent occasionally finds a better embedding scheme, it gets a lot of rewards and flexibly adjusts its behavioral strategy based on the reward results. Therefore, the performance of the four indicators training continues to improve. However, in the later stage of the training process, due to the limitations of network resources and algorithms, the performance of the learning agent stops improving. Eventually, the four indicators of the agent are stable in a range.

It can be seen from Fig. 5 that the algorithm we proposed can effectively solve the VNE problem. To further verify the performance of the algorithm proposed in this article, we use the MP-VNE algorithm proposed in the literature [36], the BASE-VNE algorithm proposed in the literature [37], the RL-VNE algorithm proposed in [38], and the SA-RL-VNE algorithm proposed in [39] as the comparison algorithm to compare with it. The baseline algorithm first sorts the candidate nodes according to node resources and then completes the embedding. For fairness, we use the same newly generated data set in the testing phases of the five algorithms.

Experiment 2 (The Latency Performance Comparisons of Five Algorithms): Fig. 6 shows the trend of network latency with running time. In this experiment, 1000 virtual network requests were set, and the latency performance was evaluated according to (4). According to the experimental data, it can be seen that, compared with DDRL-VNE, the cumulative latency of the other four algorithms is higher and the growth is faster. The main reason is that we use DRL in the node embedding

phase to improve the quality of node selection and further optimize the latency by considering the relationship between traffic and latency. The main reason is that we use DRL to improve the quality of node selection in the node embedding stage and consider the latency factor in the reward function, while other algorithms do not pay much attention to delay. In line with the analysis of specific experimental data, the cumulative latency of DDRL-VNE is 22.37% lower than SA-RL-VNE, 31.92% lower than RL-VNE, 43.48% lower than BASE-VNE, and 45.27% lower than MP-VNE.

Experiment 3 (VNR Acceptance Rate of Five Algorithms): Fig. 7 shows the trend of VNRs acceptance rate with running time. The VNR acceptance rates of all five algorithms decrease with time, which is consistent with the actual situation. The initial physical network is rich in resources and can carry a large number of VNR requests. However, with the continuous embedding of virtual networks, the available substrate network resources continue to decrease. As a result, the VNR acceptance rate of VNR gradually decreases. After adding latency constraints to the link embedding process, the schemes given by the BASE-VNE algorithm and the MP-VNE algorithm cannot carry more links, so the VNR acceptance rate decreases significantly. Although the SA-RL-VNE algorithm and RL-VNE algorithm improve the quality of candidate nodes by reinforcement learning, they do not consider the latency when extracting the feature matrix. Therefore, our proposed algorithm gives a scheme that achieves a higher success rate. In line with the analysis of specific experimental data, the acceptance rate of DDRL-VNE is 1.93% higher than SA-RL-VNE, is

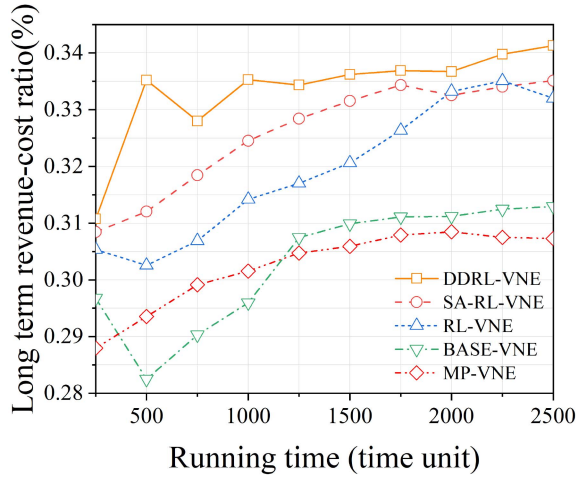


Fig. 8. Comparison of long-term average revenue-cost ratio.

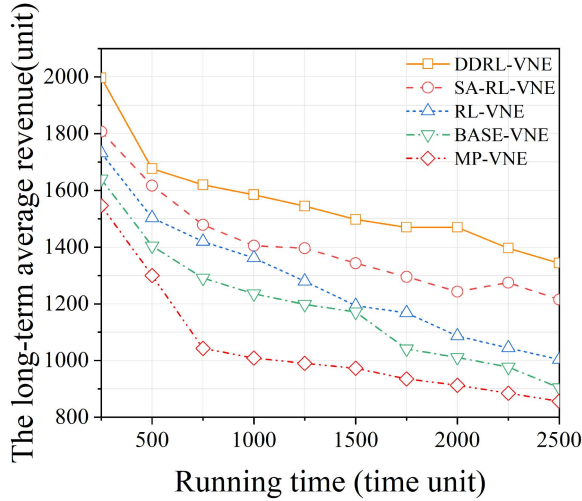


Fig. 9. Comparison of long-term average revenue.

4.06% higher than RL-VNE, 9.32% higher than BASE-VNE, and 17.64% higher than MP-VNE.

Experiment 4 (The Long-Term Average Revenue-Cost Ratio of Five Algorithms): Fig. 8 shows the changing trend of the long-term average revenue-cost ratio with running time. It can be seen from the experimental data that the long-term average revenue-cost ratio does not change significantly. This is mainly because the revenue-cost ratio is not affected by the consumption of physical resources. Our algorithm takes into account the node's CPU resources when extracting network attributes, and also takes into account the revenue in the reward function. The learning agent can obtain greater rewards in high-revenue embedding schemes. Therefore, DDRL-VNE also performs very well in terms of the revenue-cost ratio. In line with the analysis of specific experimental data, the revenue-cost ratio of DDRL-VNE is 1.84% higher than SA-RL-VNE, is 2.79% higher than RL-VNE, 9.06% higher than BASE-VNE, and 11.07% higher than MP-VNE.

Experiment 5 (The Long-Term Average Revenue of Five Algorithms): Fig. 9 shows the changing trend of the long-term average revenue of five algorithms. All five algorithms

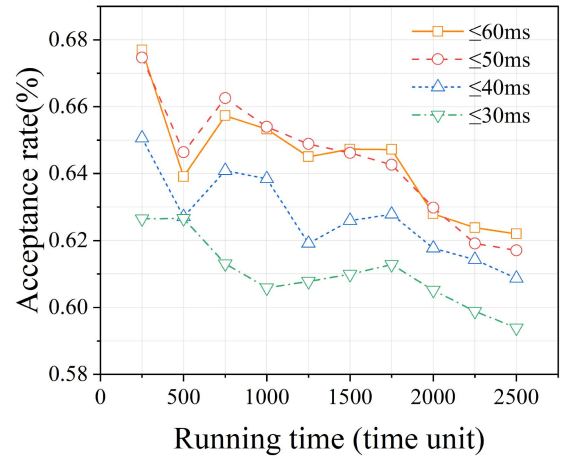


Fig. 10. Comparison of VNRs acceptance rate (take the upper limit of latency requirement as a factor).

have high long-term average revenue in the first 1000 units due to the abundance of CPU and bandwidth resources in the early stage. As nodes and links are occupied, fewer physical resources are available. This causes the substrate resources to struggle to carry the newly arriving VNRs. Therefore, the long-term average revenue decreases dramatically. DDRL-VNE uses the weighted values of revenue and latency as reward signals, and uses DRL to schedule SAGIN resources more rationally and, therefore, performs better than other algorithms in terms of long-term average revenue.

Experiment 6 (The Long-Term Average Revenue of Five Algorithms): In Experiment 6, we use the upper limit of latency requirement as a variation factor to observe the change in model performance. From Fig. 10, the changes in algorithm performance are similar to the test results, as expected. The decrease in the upper limit of delay leads to a smaller number of physical links that can satisfy the embedding requirement, causing a decline in the success rate. However, our algorithm can still maintain a high VNR acceptance rate in the face of higher latency requirements. Therefore, our algorithm can adapt to the time-varying characteristics of SAGIN and has good robustness.

The above experimental results demonstrate that our VNE algorithm can effectively sense the QoS requirements of VNRs and give a promising network resource allocation strategy. In addition, our algorithm has an excellent performance in terms of latency, long-term average revenue, and VNR acceptance rate.

VI. CONCLUSION

From the perspective of a virtual network architecture, a DRL-based latency-oriented VNE algorithm was proposed to solve the latency problem of IIoT. In DRL, we constructed a five-layer policy network as a learning agent. Under the premise of satisfying the constraints, the VNE is completed with the goal of minimizing the latency. To meet the demand for low latency in various industrialized scenarios, we additionally considered the impact of link traffic and the number of hops on latency, and significantly optimized the latency

performance of the algorithm. Experimental results showed that our proposed algorithm can arrange network resources in IIoT more flexibly and reduce network latency in a targeted manner. The performance of VNE latency is improved by 45.27%. However, the network situation of IIoT in SAGIN is extremely complicated. In future work, we will consider the delay of node forwarding and task scheduling to make the agent's training environment more realistic.

REFERENCES

- [1] Y. Liu, X. Yuan, Z. Xiong, J. Kang, X. Wang, and D. Niyato, "Federated learning for 6G communications: Challenges, methods, and future directions," *China Commun.*, vol. 17, no. 9, pp. 105–118, Sep. 2020.
- [2] H. Yang et al., "Lead federated neuromorphic learning for wireless edge artificial intelligence," *Nat. Commun.*, vol. 13, no. 1, p. 4269, Jul. 2022.
- [3] P. Zhang, C. Wang, N. Kumar, and L. Liu, "Space-air-ground integrated multi-domain network resource orchestration based on virtual network architecture: A DRL method," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 3, pp. 2798–2808, Mar. 2022.
- [4] R. Chaudhary, N. Kumar, and S. Zeadally, "Network service chaining in fog and cloud computing for the 5G environment: Data management and security challenges," *IEEE Commun. Mag.*, vol. 55, no. 11, pp. 114–122, Nov. 2017.
- [5] F. Qi, G. Mang, S. Zhang, and L. Liu, "A multi-layer architecture for space-air-ground network and IoT services," in *Proc. 17th Int. Wireless Commun. Mobile Comput. (IWCMC)*, Harbin City, China, Aug. 2021, pp. 1809–1813.
- [6] X. Chen et al., "Information freshness-aware task offloading in air-ground integrated edge computing systems," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 1, pp. 243–258, Jan. 2022.
- [7] C. Jiang and Z. Li, "Decreasing big data application latency in satellite link by caching and peer selection," *IEEE Trans. Netw. Sci. Eng.*, vol. 7, no. 4, pp. 2555–2565, Oct.–Dec. 2020.
- [8] J. Kang, Z. Xiong, D. Niyato, D. Ye, D. I. Kim, and J. Zhao, "Toward secure blockchain-enabled Internet of Vehicles: Optimizing consensus management using reputation and contract theory," *IEEE Trans. Veh. Technol.*, vol. 68, no. 3, pp. 2906–2920, Mar. 2019.
- [9] R. S. Bali and N. Kumar, "Secure clustering for efficient data dissemination in vehicular cyber-physical systems," *Future Gener. Comput. Syst.*, vol. 56, pp. 476–492, Mar. 2016.
- [10] P. Zhang, H. Sun, J. Situ, C. Jiang, and D. Xie, "Federated transfer learning for IIoT devices with low computing power based on blockchain and edge computing," *IEEE Access*, vol. 9, pp. 98630–98638, 2021.
- [11] X. Liu, C. Sun, M. Zhou, C. Wu, B. Peng, and P. Li, "Reinforcement learning-based Multislot double-threshold spectrum sensing with Bayesian fusion for industrial big spectrum data," *IEEE Trans. Ind. Informat.*, vol. 17, no. 5, pp. 3391–3400, May 2021.
- [12] C. Zhou et al., "Deep reinforcement learning for delay-oriented IoT task scheduling in SAGIN," *IEEE Trans. Wireless Commun.*, vol. 20, no. 2, pp. 911–925, Feb. 2021.
- [13] R. Chaudhary, G. S. Aujla, N. Kumar, and J. J. P. C. Rodrigues, "Optimized big data management across multi-cloud data centers: Software-defined-network-based analysis," *IEEE Commun. Mag.*, vol. 56, no. 2, pp. 118–126, Feb. 2018.
- [14] C. Wang, R. S. Batth, P. Zhang, G. S. Aujla, Y. Duan, and L. Ren, "VNE solution for network differentiated QoS and security requirements: From the perspective of deep reinforcement learning," *Computing*, vol. 103, no. 6, pp. 1061–1083, 2021.
- [15] P. Zhang, C. Wang, C. Jiang, N. Kumar, and Q. Lu, "Resource management and security scheme of ICPSS and IoT based on VNE algorithm," *IEEE Internet Things J.*, vol. 9, no. 22, pp. 22071–22080, Nov. 2022.
- [16] J. Wang, C. Jiang, H. Zhang, Y. Ren, K.-C. Chen, and L. Hanzo, "Thirty years of machine learning: The road to Pareto-optimal wireless networks," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 3, pp. 1472–1514, 3rd Quart., 2020.
- [17] P. Zhang, Y. Hong, N. Kumar, M. Alazab, M. D. Alshehri, and C. Jiang, "BC-EdgeFL: A defensive transmission model based on blockchain-assisted reinforced federated learning in IIoT environment," *IEEE Trans. Ind. Informat.*, vol. 18, no. 5, pp. 3551–3561, May 2022.
- [18] H. Yang, Z. Xiong, J. Zhao, D. Niyato, L. Xiao, and Q. Wu, "Deep reinforcement learning-based intelligent reflecting surface for secure wireless communications," *IEEE Trans. Wireless Commun.*, vol. 20, no. 1, pp. 375–388, Jan. 2021.
- [19] Y. Liu, J. Nie, X. Li, S. H. Ahmed, W. Y. B. Lim, and C. Miao, "Federated learning in the sky: Aerial-ground air quality sensing framework with UAV swarms," *IEEE Internet Things J.*, vol. 8, no. 12, pp. 9827–9837, Jun. 2021.
- [20] Z. Xiong, Y. Zhang, D. Niyato, R. Deng, P. Wang, and L.-C. Wang, "Deep reinforcement learning for mobile 5G and beyond: Fundamentals, applications, and challenges," *IEEE Veh. Technol. Mag.*, vol. 14, no. 2, pp. 44–52, Jun. 2019.
- [21] X. Ding, Z. Zhang, and D. Liu, "Low-delay secure handover for space-air-ground integrated networks," in *Proc. IEEE 31st Annu. Int. Symp. Pers. Indoor Mobile Radio Commun.*, 2020, pp. 1–6.
- [22] J. Wang, C. Jiang, K. Zhang, X. Hou, Y. Ren, and Y. Qian, "Distributed Q-learning aided heterogeneous network association for energy-efficient IIoT," *IEEE Trans. Ind. Informat.*, vol. 16, no. 4, pp. 2756–2764, Apr. 2020.
- [23] N.-N. Dao, D.-N. Vu, W. Na, J. Kim, and S. Cho, "SGCO: Stabilized green Crosshaul orchestration for dense IoT offloading services," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 11, pp. 2538–2548, Nov. 2018.
- [24] Y. Liu et al., "Deep anomaly detection for time-series data in industrial IIoT: A communication-efficient on-device federated learning approach," *IEEE Internet Things J.*, vol. 8, no. 8, pp. 6348–6358, Apr. 2021.
- [25] X. Cheng et al., "Space/aerial-assisted computing offloading for IIoT applications: A learning-based approach," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 5, pp. 1117–1129, May 2019.
- [26] X. Li, W. Feng, J. Wang, Y. Chen, N. Ge, and C.-X. Wang, "Enabling 5G on the ocean: A hybrid satellite-UAV-terrestrial network solution," *IEEE Wireless Commun.*, vol. 27, no. 6, pp. 116–121, Dec. 2020.
- [27] W. Sun, L. Wang, P. Wang, and Y. Zhang, "Collaborative blockchain for space-air-ground integrated networks," *IEEE Wireless Commun.*, vol. 27, no. 6, pp. 82–89, Dec. 2020.
- [28] A. Song, W.-N. Chen, T. Gu, H. Yuan, S. Kwong, and J. Zhang, "Distributed virtual network embedding system with historical archives and set-based particle swarm optimization," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 51, no. 2, pp. 927–942, Feb. 2021.
- [29] Z. Li, Z. Lu, S. Deng, and X. Gao, "A self-adaptive virtual network embedding algorithm based on software-defined networks," *IEEE Trans. Netw. Service Manag.*, vol. 16, no. 1, pp. 362–373, Mar. 2019.
- [30] R. Chai, D. Xie, L. Luo, and Q. Chen, "Multi-objective optimization-based virtual network embedding algorithm for software-defined networking," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 1, pp. 532–546, Mar. 2020.
- [31] G. Rui-Wen and L. Han-Cheng, "Virtual network embedding algorithm for multi-domain SDN networks," *J. Chin. Comput. Syst.*, vol. 37, no. 12, pp. 2593–2597, 2016.
- [32] H. Wu et al., "Resource management in space-air-ground integrated vehicular networks: SDN control and AI algorithm design," *IEEE Wireless Commun.*, vol. 27, no. 6, pp. 52–60, Dec. 2020.
- [33] P. Zhang, Y. Wang, N. Kumar, C. Jiang, and G. Shi, "A security and privacy-preserving approach based on data disturbance for collaborative edge computing in social IoT systems," *IEEE Trans. Comput. Social Syst.*, vol. 9, no. 1, pp. 97–108, Feb. 2022.
- [34] Z. Zhizhong, Z. Lei, F. Linlin, and L. Lilan, "Virtual network embedding in cross-domain network based on multi-dimensional attributes," *J. Univ. Electron. Sci. Technol. China*, vol. 48, no. 2, pp. 174–181, 2019.
- [35] S. Guo, Y. Dai, S. Xu, X. Qiu, and F. Qi, "Trusted cloud-edge network resource management: DRL-driven service function chain orchestration for IIoT," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6010–6022, Jul. 2020.
- [36] P. Zhang, C. Wang, Z. Qin, and H. Cao, "A multidomain virtual network embedding algorithm based on multiobjective optimization for Internet of drones architecture in industry 4.0," *Softw. Pract. Exp.*, vol. 52, no. 3, pp. 710–728, 2022.
- [37] M. Yu, Y. Yi, J. Rexford, and M. Chiang, "Rethinking virtual network embedding: Substrate support for path splitting and migration," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 38, no. 2, pp. 17–29, Apr. 2008.
- [38] H. Yao, X. Chen, M. Li, P. Zhang, and L. Wang, "A novel reinforcement learning algorithm for virtual network embedding," *Neurocomputing*, vol. 284, pp. 1–9, Apr. 2018.
- [39] P. Zhang, C. Wang, C. Jiang, and A. Benslimane, "Security-aware virtual network embedding algorithm based on reinforcement learning," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 1095–1105, Apr.–Jun. 2021.



Peiying Zhang (Member, IEEE) received the Ph.D. degree from the School of Information and Communication Engineering, University of Beijing University of Posts and Telecommunications, Beijing, China, in 2019.

He is currently an Associate Professor with the College of Computer Science and Technology, China University of Petroleum (East China), Dongying, China. He has published multiple IEEE/ACM Transactions/Journal/Magazine papers since 2016, such as the IEEE TRANSACTIONS ON INDUSTRIAL

INFORMATICS, IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS, IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY, IEEE TRANSACTIONS ON NETWORK SCIENCE AND ENGINEERING, IEEE TRANSACTIONS ON NETWORK AND SERVICE MANAGEMENT, IEEE TRANSACTIONS ON EMERGING TOPICS IN COMPUTING, IEEE NETWORK, IEEE INTERNET OF THINGS JOURNAL, *ACM Transactions on Asian and Low-Resource Language Information Processing*, *Computer Communications*, and *IEEE Communications Magazine*. His research interests include semantic computing, future Internet architecture, network virtualization, and artificial intelligence for networking.

Dr. Zhang served as the Technical Program Committee Member for IEEE ICC23, IEEE ICC22, DPPR 2021, ICIST2022, ISCIT 2016, ISCIT 2017, ISCIT 2018, ISCIT 2019, Globecom 2022, Globecom 2021, Globecom2019, COMNETSAT 2020, ICICoS 2022, SoftIoT 2021, IWCMC-Satellite 2019, IWCMC-Satellite 2020, IWCMC-Satellite 2022, and INFOCOM Wireless-Sec 2023. He is the Leading Guest Editor of *Drones*, *Mathematics*, *Electronics*, *Wireless Communications and Mobile Computing*, *Mathematical Problems in Engineering*, and *Frontiers in Psychiatry* and the editorial board of *Mobile Information Systems* and *Artificial Intelligence and Applications*.



Neeraj Kumar (Senior Member, IEEE) received the Ph.D. degree in computer science and engineering from Shri Mata Vaishno Devi University, Katra, India, in 2009.

He is currently working as a Full Professor with the Department of Computer Science and Engineering, Thapar Institute of Engineering and Technology (Deemed to be University), Patiala, India. He is also an Adjunct Professor with King Abdulaziz University, Jeddah, Saudi Arabia; Newcastle University, Newcastle upon Tyne, U.K.;

and Lebanese American University, Beirut, Lebanon. He has published more than 400 technical research papers in top-cited journals and conferences, which are cited more than 35 000 times from well-known researchers across the globe with the current H-index of 105. He has also edited/authored ten books with international/national publishers, such as IET, Springer, Elsevier, and CRC, such as *Security and Privacy of Electronic Healthcare Records: Concepts, Paradigms and Solutions*, *Machine Learning in Cognitive IoT* (CRC Press), and *Multimedia Big Data Computing for IoT Applications: Concepts* (IC4S 2019). One of the edited text-book titled *Multimedia Big Data Computing for IoT Applications: Concepts, Paradigms and Solutions* published by Springer in 2019 is having 3.5 million downloads till June 2020. It attracts the attention of researchers across the globe. His broad research areas are green computing and network management, IoT, big data analytics, deep learning, and cybersecurity.

Prof. Kumar is a Highly Cited Researcher from WoS from 2019 to 2021. He has organized various special issues of journals of repute from IEEE, Elsevier, and Springer. He has been the Workshop Chair of IEEE GLOBECOM 2018, IEEE INFOCOM 2020, and IEEE ICC 2020, and the Track Chair. He is serving as an Editor for the *ACM Computing Surveys*, *IEEE TRANSACTIONS ON SUSTAINABLE COMPUTING*, *IEEE TRANSACTIONS ON NETWORK AND SERVICE MANAGEMENT*, *Computer Communications* (Elsevier), and *International Journal of Communication Systems* (Wiley).



Ching-Hsien Hsu (Senior Member, IEEE) received the Ph.D. degree in information engineering from Feng Chia University, Taichung, Taiwan, in 1999.

He is a Chair Professor and the Dean of the College of Information and Electrical Engineering, Asia University, Taichung; a Professor with the Department of Computer Science and Information Engineering, National Chung Cheng University, Chiayi, Taiwan; and Research Consultant, Department of Medical Research, China Medical University Hospital, China Medical University, Taichung.

He has been acting as an author/coauthor or an editor/co-editor of ten books from Elsevier, Springer, IGI Global, World Scientific, and McGraw-Hill. His research includes cloud and edge computing, big data analytics, high-performance computing systems, parallel and distributed systems, artificial intelligence, medical AI, and natural language processing. He has published more than 350 papers in top journals, such as the IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, IEEE TRANSACTIONS ON SERVICES COMPUTING, *ACM Transactions on Multimedia Computing, Communications, and Applications*, IEEE TRANSACTIONS ON CLOUD COMPUTING, IEEE TRANSACTIONS ON EMERGING TOPICS IN COMPUTING, IEEE SYSTEM, and IEEE NETWORK, top conferences, and book chapters in these areas.

Dr. Hsu was awarded seven times Talent Awards from the Ministry of Science and Technology and Ministry of Education, and nine times Distinguished Award for excellence in research from Chung Hua University, Hsinchu, Taiwan. He is the Editor-in-Chief of the *International Journal of Grid and High Performance Computing* and *International Journal of Big Data Intelligence* and serves as an editorial board member for a number of prestigious journals. Since 2008, he has been serving as an Executive Committee Member for the IEEE Technical Committee of Scalable Computing, IEEE Special Technical Committee Cloud Computing, and Taiwan Association of Cloud Computing. He is a Fellow of the IET (IEE) and the Chair of the IEEE Technical Committee on Cloud Computing.



Yi Zhang is currently pursuing the master's degree with the College of Computer Science and Technology, China University of Petroleum (East China), Dongying, China.

His research interests include network virtualization and artificial intelligence for networking.